Relay: reasoning & approach

Chrome screen recorder and local workspace

Naman Madhogaria | Version 0.4.0 | 11 October 2026

1. The assignment and my focus

The assignment was to build a Chrome extension similar to Loom, with a better user experience, while showing thoughtful AI use and scalable application development. I focused on a clear core journey: choose a source, record, review the result, and organize the video.

My aim was to reduce friction through optional microphone and camera setup, a microphone check, predictable recording controls, immediate preview, and local saving without sign-in. This submission combines local recording with optional AWS backups, not a complete replacement for Loom.

2. Product flow and UX decisions

Prepare: The user opens Recording studio, chooses microphone and camera options, and selects a screen, window, or tab through Chrome. Browser prompts keep capture under the user's control.

Record: A countdown leads into recording. Pause and resume keep one clip together. The studio runs in a full browser tab, so closing the small extension launcher does not end capture.

Review: After finishing, the user can preview, rename, and download the video. New recording returns to setup while keeping the saved library. Save failures leave preview and download available.

Organize: Library, favorites, folders, tags, and playlists help users find recordings. Settings include a saved display name and light/dark appearance. Empty states and statistics reflect actual local data.

3. Scope and access

All thirteen pages are available without sign-in or admin roles. Management tools apply to the current installation; they do not manage a shared online team. Each Chrome profile has a local workspace and optional guest cloud backup space. Billing was intentionally omitted.

The final release does not include captions, translation, public sharing links, meeting auto-capture, or cross-device sync. AI assisted development, but there is no AI-powered end-user feature in this release.

Technical decisions

How recording, camera composition, and local persistence work

4. Technology choices

React and TypeScript: React provides reusable interface components. TypeScript helps catch inconsistent data and state usage before runtime. Vite bundles the app and its assets for the Chrome extension.

Browser media APIs and MediaRecorder: Chrome supplies user-selected screen capture and device access. MediaRecorder creates the WebM video. Codec support is checked at runtime. Recorded audio is microphone audio; system and tab audio are not implemented.

IndexedDB: Completed video blobs are stored in the browser database. The app waits for transaction completion before reporting a successful save. Small settings and workspace metadata use localStorage.

5. Camera in the actual video

With the camera enabled, a canvas combines the screen with a circular crop of the camera. The canvas video track and microphone audio track are passed to MediaRecorder, embedding the face in the saved file. Screen-only recording uses the direct capture path.

A worker drives drawing to avoid relying on animation callbacks that can pause in hidden tabs. Camera-composed output is bounded to 1080p, with a fixed overlay position to keep complexity and processing cost manageable. Real hardware and background behavior still require manual checks.

A separate floating camera window lets the presenter see themselves while using another application. Entire-screen capture can include that window as well as the embedded camera. Hiding the floating preview avoids this duplication without removing the saved overlay.

6. Local data and privacy

Videos live in IndexedDB under relay-recordings. Workspace metadata and the theme are stored locally. Playlists reference videos instead of duplicating the files. Deleting a recording removes its playlist references and clears linked meeting references while keeping the agendas.

The extension permits network access only to its configured S3 and Cognito endpoints. Cloud uploads are explicit. Temporary credentials remain in memory. Camera and microphone use browser permission prompts. Keep the Chrome profile and extension storage: clearing it may lose local videos and the guest identity needed for cloud access.

Workspace metadata export contains organization details, not the video files, and is not a restorable full backup.

Challenges & AI workflow

What required care, how AI was used, and what changed during development

7. Key challenges and responses

Capture lifecycle: Permission requests can finish after a user cancels or stops sharing. State checks and cleanup prevent late streams from remaining active. Media resources are released on stop, cancellation, failure, and relevant navigation.

Camera reliability: A camera visible on the page is not automatically part of the recording. Canvas composition solves this. Camera disconnection stops the clip instead of silently recording a frozen face.

Long recordings: Removing the fixed duration cap improves flexibility, but does not make memory unlimited. Media chunks accumulate until recording ends. Available RAM and storage remain limits; crash recovery and multi-hour endurance are not guaranteed.

Accurate product behavior: Insights count local playback sessions rather than public or unique viewers. Meetings store agendas and export calendar files; they do not join calls or send invitations. These distinctions keep the interface and submission claims accurate.

8. Thoughtful AI-assisted development

I used Codex to help scaffold components and recording logic, refactor code, develop focused tests, automate browser checks, and prepare documentation. I checked generated work through TypeScript, automated tests, browser interactions with synthetic media, and visual review.

The important distinction is between generating code and proving behavior. Automated browser checks do not prove that every laptop microphone, camera, permission flow, or long recording works. Those checks are documented separately for real-device review.

9. Working AWS backup integration

Amazon S3 in Mumbai stores selected recording backups. Amazon Cognito issues temporary guest credentials, and IAM scopes each guest to its own folder. CloudFormation defines the deployed relay-cloud stack. The Cloud backups page supports upload, listing, restore to Library, and independent cloud deletion without a Relay login.

The bucket uses encryption, HTTPS, public-access blocking, versioning, and access logs. Multipart uploads limit extra transfer buffering. The app permits up to 1 GB per cloud video; this is not an AWS spending cap. Captions remain removed after the earlier account rejected AI processing. AWS backup works independently of captions.

This release demonstrates AI-assisted development. It does not fully meet an interpretation of the assignment that requires a working AI product feature. I have kept that gap explicit rather than presenting blocked or simulated captions as complete.

Validation & next steps

Evidence, scalability boundaries, and the reviewer handoff

10. Verification completed

The recorded release checks include a successful TypeScript/Vite production build, 52 frontend tests, and six backend tests. Frontend coverage includes permission recovery, timing, camera cleanup, storage, navigation, theme, search, playlists, deletion references, and calendar handling.

A separate Chrome profile used a real WebM generated from synthetic media. Browser checks covered recording workflows, organization, dark mode, and a 390-pixel viewport. Live AWS tests uploaded and restored exact video bytes, checked multipart transfer, and verified another guest could not read, overwrite, delete, or list the files. Screenshots were visually inspected.

Still separate: A real-device checklist covers microphone sound, camera motion, downloaded playback, denied permissions, Chrome Stop sharing, switching applications, and reopening saved data. These are not claimed as passed by the synthetic-media checks.

11. Scalability and improvements

The code separates interface, capture, composition, persistence, and cloud transfer. S3 and Cognito supply managed storage and access without an always-running app server. This does not yet provide a shared team workspace.

First priority: Persist small recording chunks during capture and add a verified recovery flow. This would reduce memory pressure and improve resilience. Add paginated metadata queries as libraries grow.

Later: Wider distribution needs enforced storage quotas, abuse controls, cost monitoring, and authenticated recovery. Guest access is not account recovery. AI captions should return only after the service is available and evaluated with mixed-language audio. These are future improvements.

12. Deliverables and supporting files

The handoff includes the source code, ready-to-load extension ZIP, this reasoning document, setup instructions, and testing notes. The product demo video is a separate deliverable to record using the prepared demo script.

Repository: github.com/NamanMadhogaria/relay
Release: Relay v0.4.0 packages
The repository is private; reviewers need access or the submission ZIP.

Consolidated from SUBMISSION.md, docs/reasoning.md, and docs/testing.md. Technical references used during development include Chrome screen-capture and Document Picture-in-Picture documentation, MDN MediaRecorder, Vite documentation, and AWS Cloudscape typography guidance.